

Ecosistema de Automatización IA Autónomo

Triage inteligente de tickets de soporte con Human-in-the-Loop

Campo	Detalle
Caso de negocio	Mesa de ayuda de NimbusCRM (SaaS de gestión de contactos)
Orquestador	n8n Cloud — workflows TCK-01 y TCK-99
Base de datos	Airtable — base «Mesa de Ayuda IA», 4 tablas vinculadas
Procesamiento IA	Cohere Chat API v2 — Command R7B y Command A
Canal de salida	Gmail (cliente final) + Telegram (aprobación y alertas internas)
Nodos totales	30 en TCK-01 + 4 en TCK-99

Enlaces de la entrega

Recurso	Enlace
Repositorio GitHub	https://github.com/Javs-Arbulu/Entrega_Final
Base de datos Airtable (modo lectura)	https://airtable.com/app6VZNu6Sf0z8moD/shrwGB7MMp4VjjZk0

Autor: Javier Arbulú

Resumen ejecutivo

Este documento describe un ecosistema de automatización que resuelve de extremo a extremo el triage y la respuesta de tickets de soporte. Un formulario web dispara un webhook; el sistema normaliza y valida el payload, persiste al cliente y al ticket en Airtable, clasifica la consulta con un modelo de lenguaje, recupera los artículos aplicables de una base de conocimiento, redacta una respuesta con un segundo modelo, solicita aprobación humana por Telegram y, sólo si un revisor aprueba, envía el correo al cliente y registra la telemetría de la corrida.

El sistema opera sin intervención manual salvo en el punto de control humano, que es deliberado. Incorpora cuatro rutas de error a nivel de nodo, un workflow independiente de captura global, un filtro anti-bucle infinito y una política de minimización de datos que impide que información personal identificable llegue al proveedor de IA.

Cumplimiento de los criterios de evaluación

#	Criterio	Dónde se resuelve
1	Mapa de arquitectura	Sección 1 y diagrama apaisado de la página siguiente
2	Estructuras de datos documentadas	Sección 2 y carpeta /schemas del repositorio
3	Optimización de costos	Sección 3: matriz de decisión por tarea y cálculo de ahorro
4	Seguridad y resiliencia	Sección 4: minimización, error handlers y HITL
5	Dashboard de control	Sección 5 y enlace público a la Shared View de Airtable

Tecnologías integradas (las cuatro categorías obligatorias)

Categoría	Implementación	Rol en el sistema
Orquestador	n8n Cloud	Ejecuta la lógica, el ruteo y la gestión de errores
Base de datos	Airtable	Memoria del sistema, máquina de estados y telemetría
Procesamiento IA	Cohere Chat API v2	Clasificación estructurada y redacción sobre contexto recuperado
Canal de salida	Gmail + Telegram	Respuesta al cliente y canal de aprobación interno

Sobre el canal de salida. La consigna admite Gmail, Slack o WhatsApp API. Se eligió Gmail como canal hacia el cliente final, que es el que cumple el requisito. Telegram se incorpora como canal interno adicional para el punto de aprobación humana y las alertas de error, por tres ventajas técnicas frente a WhatsApp: no impone ventana de 24 horas, no exige plantillas aprobadas por el proveedor, y su nodo en n8n soporta la operación `sendAndWait`, que suspende la ejecución hasta recibir la respuesta del revisor.

1 · Mapa de arquitectura

El diagrama completo del sistema está en la página siguiente, en orientación apaisada. Reproduce los 30 nodos de TCK-01 con sus nombres exactos, de modo que pueda cotejarse directamente contra el archivo `TCK-01_completo.json` del repositorio, más los 4 nodos del workflow de captura global TCK-99.

Flujo en nueve fases

Fase	Nodos	Qué ocurre
1 · Ingesta	3	Webhook con Header Auth recibe el formulario; se normaliza el payload y se valida contra cuatro condiciones
2 · Persistencia	2	Upsert del cliente por email y alta del ticket en estado Pendiente
3 · Clasificación IA	4	Se arma el prompt, se llama a Command R7B en modo JSON, se valida la respuesta y se actualiza el ticket
4 · Ruteo	2	Un Switch descarta lo que no es soporte antes de gastar el modelo grande
5 · Recuperación KB	2	Búsqueda filtrada por categoría y plan, máximo tres artículos, consolidados en un ítem
6 · Redacción IA	4	Command A redacta sobre el contexto recuperado; se extrae el texto, el consumo real y el costo
7 · Human-in-the-loop	2	Telegram suspende la ejecución hasta que un revisor aprueba o rechaza
8 · Salida al cliente	3	Gmail envía la respuesta, se guarda el Thread ID y se registra la corrida
9 · Rechazo y reintento	3	Se registra el feedback, se reintenta una vez y luego se escala a un humano

Componentes exigidos por la rúbrica

Componente	Nodo o mecanismo
Trigger	TRG · Webhook Nuevo Ticket, POST /webhook/ticket-nuevo, protegido con Header Auth. Disparador específico por evento y no un polling periódico, para no consumir operaciones innecesarias.
Routers	SWITCH · Ruteo Categoría (descarte temprano) · IF · Validar Payload (calidad de datos) · IF · Aprobado (decisión humana) · IF · Intentos < 2 (control de bucle)
APIs externas	Cohere Chat API v2 (dos llamadas con modelos distintos) · Airtable REST (14 nodos) · Telegram Bot API (2 nodos) · Gmail API (1 nodo)
Nodos de IA	AI · Clasificar Ticket (Command R7B, salida JSON forzada) · AI · Redactar Respuesta (Command A, redacción sobre contexto)
Destino de los datos	Airtable como sistema de registro: Tickets (estado y contenido), Clientes (identidad), Base_Conocimiento (trazabilidad de qué se usó) y Log_Ejecuciones (telemetría). Gmail como destino externo.

Decisiones de arquitectura

- **Prompts en nodos Code, no en el campo del nodo HTTP.** Cada llamada a Cohere está precedida por un nodo Code que construye el cuerpo de la petición como objeto JavaScript. El nodo HTTP sólo ejecuta `JSON.stringify($json.body)`. Esto evita errores de escapado de comillas y saltos de línea dentro de un JSON embebido, y deja el prompt versionado y legible.
- **El bucle de reintento vuelve al nodo de prompt, no al nodo HTTP.** Es donde vive la condición `$runIndex > 0` que inyecta la instrucción de reescribir con otro enfoque cuando el revisor rechazó el borrador anterior.
- **Referencias por nombre de nodo en lugar de \$json.** Las expresiones usan `$('Nodo').first()`, lo que hace el flujo resistente a reordenamientos y evita los errores de emparejamiento de ítems típicos cuando hay llamadas a IA en el medio de la cadena.

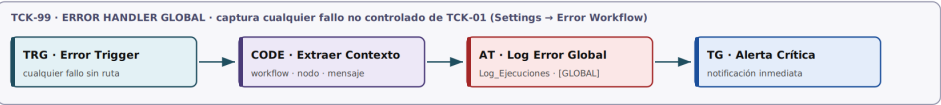
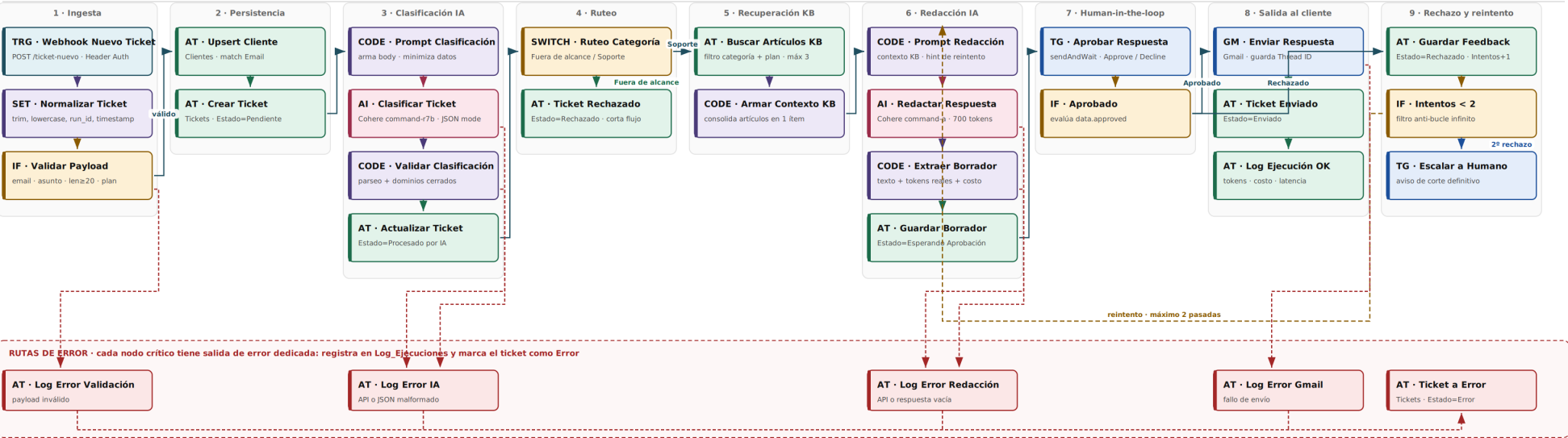
- **Respuesta inmediata del webhook.** El trigger responde antes de que el flujo termine. Es obligatorio: la fase 7 puede permanecer suspendida hasta 24 horas esperando al revisor, y ningún cliente HTTP mantendría esa conexión abierta.

1 · Mapa de arquitectura del sistema

TCK-01 (30 nodos) y TCK-99 (4 nodos). Ampliar para leer el detalle de cada nodo.

TCK-01 · Triage de Tickets de Soporte con IA

Ecosistema de automatización · n8n (orquestador) · Airtable (memoria) · Cohere (razonamiento) · Telegram + Gmail (canales)



REFERENCIAS

- Trigger / entrada
- Transformación (Code / Set)
- Lógica (IF / Switch)
- Airtable
- Motor de IA (Cohere)
- Canal de salida
- Ruta de error

- PUNTOS DE CONTROL EXIGIDOS
- ✓ Human-in-the-loop en fase 7: el flujo espera aprobación
 - ✓ 4 rutas de error por nodo + handler global TCK-99
 - ✓ Filtro anti-bucle infinito: máximo 2 intentos
 - ✓ Minimización: sin datos personales hacia el modelo

2 · Estructuras de datos documentadas

El sistema utiliza Airtable como memoria y registro. Cuatro tablas vinculadas evitan datos aislados: **Tickets** es la tabla central y enlaza con **Clientes** (quién consulta), **Base_Conocimiento** (qué material se usó para responder) y **Log_Ejecuciones** (qué ocurrió en cada corrida).

2.1 · Modelo relacional

Relación	Cardinalidad	Campo que la materializa	Para qué sirve
Clientes a Tickets	1 a N	Cliente (link)	Historial por cliente sin duplicar identidad
Base_Conocimiento y Tickets	N a N	Artículos KB (link)	Trazabilidad: qué artículos fundamentaron cada respuesta
Tickets a Log_Ejecuciones	1 a N	Ticket (link)	Telemetría por corrida, incluidos los reintentos

La deduplicación de clientes se resuelve con la operación upsert del nodo de Airtable usando el email como columna de coincidencia. Sin ella, cada corrida del test de estrés crearía un cliente nuevo.

2.2 · Tabla Tickets (central)

Campo	Tipo	Origen	Notas
Ticket ID	Formula	Airtable	Campo primario: "TCK-" & RIGHT("0000" & N, 4)
N	Autonumber	Airtable	Alimenta la fórmula anterior
Asunto / Descripción	Text / Long text	Webhook	Único contenido de la consulta
Cliente	Link a Clientes	n8n	Array de record IDs
Estado	Single select	n8n	Máquina de estados de 6 valores
Categoría	Single select	IA	5 opciones, incluida «Fuera de alcance»
Prioridad	Single select	IA	P1 / P2 / P3
Sentimiento	Single select	IA	Neutro / Frustrado / Enojado
Resumen IA	Long text	IA	Máximo 30 palabras
Confianza	Number (2 dec.)	IA	0 a 1, acotado en el nodo de validación
Borrador Respuesta	Long text	IA	Cuerpo propuesto, previo a la aprobación
Artículos KB	Link a Base_Conocimiento	n8n	Hasta 3 registros
Intentos IA	Number entero	n8n	Filtro anti-bucle infinito
Feedback Humano	Long text	HITL	Motivo del rechazo
Gmail Thread ID	Text	Gmail	Mantiene el hilo para seguimientos
Fecha Envío	Date con hora	n8n	Marca de entrega efectiva

2.3 · Máquina de estados

Recorrido	Significado
Pendiente > Procesado por IA > Esperando Aprobación > Enviado	Camino feliz completo
Pendiente > Procesado por IA > Rechazado	Descarte temprano por categoría «Fuera de alcance»
Esperando Aprobación > Rechazado > Esperando Aprobación	Reintento tras rechazo humano (una sola vez)
cualquier estado > Error	Fallo de validación, de API de IA o de envío

2.4 · Esquemas JSON de transferencia

Cinco puntos donde los datos cruzan un límite del sistema. Los archivos completos, en formato JSON Schema draft 2020-12 cuando corresponde, están en la carpeta /schemas del repositorio.

A · Entrada del webhook

n8n entrega el cuerpo de la petición anidado bajo la clave body. Todas las expresiones del nodo de normalización parten de \$json.body.

```
{ "nombre": "Laura Méndez", "email": "laura.mendez@vertexlab.pe",
  "empresa": "Vertex Lab", "plan": "Enterprise",
  "asunto": "La sincronización de contactos quedó a medias",
  "descripcion": "Corrimos la sincronización anoche y de 4.200 contactos..." }
```

Campo	Restricción validada en IF · Validar Payload
email	Debe cumplir la expresión regular <code>^[^@\\s]+@[^@\\s]+\\. [a-zA-Z]{2,}\$</code>
asunto	No vacío
descripcion	Longitud mínima de 20 caracteres, comparada como número
plan	Debe ser Free, Pro o Enterprise

B · Objeto canónico interno

El nodo SET · Normalizar Ticket produce el contrato interno que consumen todos los nodos posteriores. Aplica trim y toLowerCase al email, define Free como plan por defecto y garantiza tipo cadena en todos los campos mediante ?. y || ''. Añade dos campos propios: run_id (identificador de ejecución, correlaciona todos los registros de telemetría de una corrida) y recibido_en (base del cálculo de latencia).

C · Petición y respuesta de clasificación

```
POST https://api.cohere.com/v2/chat    Authorization: Bearer <COHERE_API_KEY>

{ "model": "command-r7b-12-2024", "temperature": 0.2, "max_tokens": 300,
  "response_format": { "type": "json_object" },
  "messages": [ { "role": "system", "content": "<prompt de triage>" },
                { "role": "user", "content": "Asunto / Descripción / Plan" } ] }
```

Cohere v2 devuelve el texto dentro de message.content como un **array de bloques**, no como cadena. El nodo de validación aplana ese array antes de intentar el parseo; es una diferencia de formato frente a OpenAI que rompe las implementaciones ingenuas.

```
{ "message": { "role": "assistant",
               "content": [ { "type": "text", "text": "{ \"categoria\": ... }" } ] },
  "usage": { "tokens": { "input_tokens": 372, "output_tokens": 118 } } }
```

Contenido útil, ya parseado:

```
{ "categoria": "Bug técnico", "prioridad": "P2", "sentimiento": "Frustrado",
  "resumen": "Sincronización parcial de contactos sin detalle de filas rechazadas.",
  "confianza": 0.92 }
```

D · Petición de redacción

Mismo endpoint, modelo Command A, max_tokens 700 y temperatura 0.4. El mensaje de usuario concatena los datos del ticket con los artículos recuperados de la base de conocimiento. Cuando la búsqueda no devuelve resultados, se envía la cadena literal SIN ARTÍCULOS RELEVANTES y el prompt obliga al modelo a escalar el caso en lugar de inventar procedimientos. Es el control de alucinación del sistema.

E · Payload de aprobación (Human-in-the-loop)

La operación sendAndWait de Telegram suspende la ejecución y publica un webhook de reanudación al que apuntan los botones del mensaje. Cuando el revisor pulsa uno, n8n reanuda el flujo con este ítem:

```
{ "data": { "approved": true } }    // Approve
{ "data": { "approved": false } }   // Decline
```

El nodo IF · Aprobado evalúa \$json.data.approved como booleano. El tiempo límite de espera es de 24 horas; vencido, la ejecución se cierra sin enviar nada al cliente y el ticket permanece en «Esperando Aprobación».

3 · Matriz de optimización de costos

El sistema no usa un único modelo. Asigna a cada tarea el modelo más barato capaz de resolverla, y descarta trabajo antes de llegar a la etapa cara. Esta sección justifica esa decisión con números.

3.1 · Tarifas de referencia

Modelo	Entrada / 1M tokens	Salida / 1M tokens	Perfil
Command R7B (12-2024)	US\$ 0,0375	US\$ 0,15	El más económico del catálogo. Diseñado para clasificación, ruteo y tareas de recuperación acotadas.
Command R (08-2024)	US\$ 0,15	US\$ 0,60	Intermedio. Alternativa si R7B se quedara corto.
Command A (03-2025)	US\$ 2,50	US\$ 10,00	Ventana de 256K y mejor desempeño en RAG y tono. 67 veces más caro en entrada que R7B.

Tarifas verificadas contra la documentación de Cohere y agregadores de precios independientes. Deben reconfirmarse en el momento de la entrega: son las constantes PRECIO_IN_POR_M y PRECIO_OUT_POR_M del nodo CODE · Extraer Borrador, y de ellas depende el campo Costo USD del dashboard.

3.2 · Matriz de decisión por tarea

Tarea	Modelo asignado	Justificación técnica	Costo por ejecución
Clasificación de ticket	Command R7B	Dominios cerrados, salida de 5 campos, sin razonamiento de varios pasos. Se fuerza el formato con response_format y se validan los valores en código, así que un modelo grande no aportaría precisión adicional.	US\$ 0,000032
Redacción de la respuesta	Command A	Debe leer hasta 3 artículos completos, respetar tono, límite de extensión y prohibición de inventar procedimientos. Es donde la calidad del modelo se traduce en calidad percibida por el cliente.	US\$ 0,004533
Reprocesamiento masivo histórico	Batch / lotes	Los tickets archivados no tienen urgencia. Procesarlos en lotes fuera de línea evita competir con el tráfico en vivo y permite negociar tarifas de volumen. No implementado en esta entrega por no aplicar al caso en vivo.	—

3.3 · Ahorro medido

Escenario A · por ticket procesado completo

Configuración	Clasificación	Redacción	Total por ticket
Arquitectura híbrida (implementada)	US\$ 0,000032	US\$ 0,004533	US\$ 0,004564
Todo con Command A	US\$ 0,002110	US\$ 0,004533	US\$ 0,006642
Ahorro			31,3 %

Escenario B · 1.000 tickets, con 15 % fuera de alcance

El descarte temprano del Switch multiplica el ahorro: un mensaje clasificado como «Fuera de alcance» nunca llega a la etapa de redacción, así que cuesta sólo la clasificación.

Configuración	Detalle del cálculo	Costo total
Arquitectura híbrida con descarte temprano	1.000 clasificaciones R7B + 850 redacciones Command A	US\$ 3,88
Command A en todo, sin descarte	1.000 clasificaciones + 1.000 redacciones con Command A	US\$ 6,64

Configuración	Detalle del cálculo	Costo total
Ahorro	Mismo resultado funcional	41,5 %

Consumos de referencia: clasificación 372 tokens de entrada y 118 de salida; redacción 505 y 327. Los valores reales de cada corrida quedan registrados en la tabla Log_Ejecuciones, porque el nodo CODE · Extraer Borrador lee el objeto usage que devuelve la API en lugar de estimarlos.

3.4 · Optimizaciones adicionales implementadas

Mecanismo	Dónde	Efecto sobre el costo
Descarte temprano	SWITCH · Ruteo Categoría	Elimina por completo la llamada al modelo caro en los mensajes que no son soporte
Límite de max_tokens	Ambos nodos de IA (300 y 700)	Acota el gasto de salida, que es el componente más caro en Command A
Máximo 3 artículos de KB	AT · Buscar Artículos KB	Sin este tope, la entrada crecería con cada artículo nuevo de la base
Minimización del prompt	CODE · Prompt Clasificación	No se envían nombre, email ni empresa: menos tokens y menos exposición
Reintento acotado a 2	IF · Intentos < 2	Techo duro al gasto por ticket; impide un bucle de rechazos indefinido
Reintentos de red limitados	retryOnFail, 2 intentos	Evita reintentos infinitos ante errores transitorios de la API

4 · Seguridad y resiliencia

4.1 · Minimización de datos

El sistema trata al proveedor de IA como un tercero no confiable. Sólo recibe lo estrictamente necesario para resolver la tarea; ningún dato personal identificable sale hacia la API.

Campo	¿Va al modelo?	Fundamento
asunto	Sí	Es el contenido a clasificar
descripcion	Sí	Es el contenido a clasificar y responder
plan	Sí	Determina qué artículos de la base de conocimiento son aplicables
nombre	No	No aporta a la clasificación ni a la redacción
email	No	Dato de contacto, innecesario para la tarea
empresa	No	Dato identificador, innecesario para la tarea

La personalización se resuelve **después** del modelo: el prompt obliga a abrir con «Hola,» sin nombre propio, y el nodo de Gmail inyecta el nombre del cliente mediante una sustitución de texto en el momento del envío. El resultado que recibe el cliente es personalizado, pero el modelo nunca conoció su identidad.

4.2 · Gestión de credenciales

Credencial	Alcance	Principio aplicado
Header Auth del webhook	Sólo autentica el formulario contra el endpoint	Cadena aleatoria dedicada, sin relación con ninguna otra credencial del sistema
Airtable PAT	Lectura y escritura de registros, lectura de esquema, limitado a una sola base	Privilegio mínimo: no se otorgan permisos sobre el resto del espacio de trabajo
Cohere API key	Endpoint de chat	Almacenada en el gestor de credenciales de n8n, nunca en el cuerpo del flujo
Telegram Bot Token	Envío y recepción del bot dedicado	Bot exclusivo del proyecto
Gmail OAuth2	Envío de correo	Autorización delegada, revocable sin rotar contraseñas

Ninguna credencial aparece en los archivos JSON exportados: n8n las almacena por referencia. El archivo del flujo puede publicarse en un repositorio público sin riesgo.

4.3 · Rutas de error

Cinco nodos críticos tienen salida de error dedicada (directiva continue using error output). Ninguna falla interrumpe la ejecución en silencio: todas se registran y todas dejan el ticket en un estado consultable.

Nodo de origen	Modo de falla	Destino	Efecto
IF · Validar Payload	Payload incompleto o mal formado	AT · Log Error Validación	Se registran todos los motivos de rechazo concatenados; no se crea el ticket
AI · Clasificar Ticket	API caída, clave inválida, límite de tasa	AT · Log Error IA	Ticket a Error tras 2 reintentos automáticos
CODE · Validar Clasificación	JSON no parseable o valor fuera de dominio	AT · Log Error IA	Impide escribir un valor inválido en un Single Select de Airtable
AI · Redactar Respuesta	API caída o límite de tasa	AT · Log Error Redacción	Ticket a Error, sin enviar nada al cliente
CODE · Extraer Borrador	Respuesta vacía del modelo	AT · Log Error Redacción	Se detecta el caso silencioso de un texto en blanco

Nodo de origen	Modo de falla	Destino	Efecto
GM · Enviar Respuesta	Fallo de envío o token expirado	AT · Log Error Gmail	El borrador aprobado queda preservado para reenvío manual

Captura global: workflow TCK-99

Un segundo workflow independiente, asignado en la configuración de TCK-01 como Error Workflow, se dispara ante cualquier fallo no contemplado por las rutas anteriores. Extrae el contexto (workflow, último nodo ejecutado, mensaje), lo registra en Log_Ejecuciones con el prefijo [GLOBAL] y envía una alerta inmediata por Telegram. La cobertura de errores es por lo tanto de dos niveles: por nodo y global.

4.4 · Punto de validación humana

El sistema se detiene antes de la única acción irreversible: enviar un correo a un cliente real. La operación `sendAndWait` del nodo de Telegram suspende la ejecución y envía al revisor el identificador del ticket, la prioridad, la categoría, el número de intento, la cantidad de artículos utilizados, la consulta original y el borrador propuesto, con dos botones.

Decisión del revisor	Consecuencia
Approve	Gmail envía la respuesta, se guarda el Thread ID y se registra la corrida con tokens, costo y latencia
Decline (primer rechazo)	Se registra el feedback, se incrementa el contador y el flujo vuelve al nodo de prompt con la instrucción de reescribir con otro enfoque
Decline (segundo rechazo)	El filtro anti-bucle corta la automatización y notifica que el ticket requiere redacción manual
Sin respuesta en 24 horas	La ejecución se cierra sin contactar al cliente; el ticket permanece en «Esperando Aprobación»

Este diseño evita el efecto metrallata: ninguna respuesta generada automáticamente llega a un cliente sin que una persona la haya leído.

4.5 · Respuesta al check de seguridad

Pregunta de control	Respuesta
¿El flujo tiene un filtro para evitar bucles infinitos?	Sí. El campo Intentos IA de Airtable se incrementa en cada rechazo y el nodo IF · Intentos < 2 corta el ciclo en el segundo. El techo es duro: no depende de la conducta del modelo ni del revisor.
¿Se comparan tipos de datos correctos en los filtros?	Sí. La validación de longitud usa <code>\$json.descripcion.length</code> con el operador en modo numérico y validación estricta activada. Con comparación de cadenas, «9» sería mayor que «20» y un texto de una palabra pasaría el filtro.
¿El prompt de IA es dinámico y usa variables del sistema?	Sí. Ambos prompts se construyen en tiempo de ejecución con datos del ticket, el plan del cliente y los artículos recuperados. El de redacción además inyecta condicionalmente una instrucción de reescritura según <code>\$runIndex</code> . No hay contenido fijo en duro.

5 · Dashboard de control

El panel se publica como vista compartida de Airtable con acceso de sólo lectura. Se alimenta de la tabla Log_Ejecuciones, que recibe un registro por cada corrida y por cada fallo, y de la tabla Tickets, que expone la distribución por estado.

5.1 · Vistas publicadas

Vista	Tabla	Configuración	Qué responde
Estado operativo	Tickets	Agrupada por Estado	Cuántos tickets hay en cada etapa y cuántos están esperando aprobación
Tasa de errores	Log_Ejecuciones	Filtrada por Resultado = Error, agrupada por Nodo	Qué proporción de corridas falla y en qué punto del flujo
Consumo y costo	Log_Ejecuciones	Filtrada por Resultado = OK, con sumatorias	Costo acumulado, tokens totales y latencia media
Calidad del triage	Tickets	Agrupada por Categoría y Prioridad	Distribución de la clasificación y proporción de fuera de alcance

5.2 · Indicadores

Indicador	Cómo se obtiene
Tickets procesados	Conteo de registros en Tickets
Tasa de aprobación humana	Enviados sobre el total que llegó a Esperando Aprobación
Tasa de reintento	Tickets con Intentos IA = 2 sobre el total
Tasa de error del sistema	Registros con Resultado = Error sobre el total de Log_Ejecuciones
Costo acumulado	Sumatoria del campo Costo USD
Costo medio por ticket	Costo acumulado dividido por tickets enviados
Latencia media de extremo a extremo	Promedio de Latencia ms, medida desde la recepción del webhook
Proporción de fuera de alcance	Tickets con categoría «Fuera de alcance» sobre el total

La latencia registrada es de extremo a extremo e incluye el tiempo de espera humana, por lo que en los tickets aprobados manualmente refleja el tiempo de respuesta del revisor más que el del sistema. Para medir sólo el procesamiento automático debe leerse el registro previo a la suspensión.

Anexo · Contenido del repositorio

Ruta	Contenido
README.md	Índice, descripción del proyecto y los cuatro enlaces obligatorios
docs/Entrega_Final.pdf	Este documento
docs/01_Diagrama_Arquitectura.pdf	Diagrama en archivo independiente
flows/TCK-01_completo.json	Workflow principal, 30 nodos, exportado desde n8n
flows/TCK-99_error_handler.json	Workflow de captura global de errores
schemas/	Seis archivos de esquema: entrada, objeto canónico, ambas llamadas a Cohere, aprobación de Telegram y esquema de Airtable
data/base_conocimiento.csv	Los diez artículos de la base de conocimiento
screenshots/	Evidencias de las corridas, incluidos los caminos infelices